

J2EE and Open Source Innovation: The Relationship between Open Source and Standards

Author: Keith Pitty

Abstract

What is the relationship between the Java Enterprise Edition standard and open source innovation? This paper attempts to answer that question.

Several examples of open source projects (Hibernate, Spring, Struts, Ruby on Rails and JBoss Seam) are used to show how open source projects have provided innovative solutions to challenges faced during the history of the Java EE (formerly J2EE) specification.

The examples also demonstrate that open source innovation in some cases has led, and continues to lead, directly to improvements in the Java EE specification. In others it has challenged the Java EE community to debate the merits of standards based development compared with using open source frameworks and libraries.

The Java and Open Source Communities

Before exploring the relationship between the Java Enterprise Edition standard and open source innovation it is useful to provide some broad historical context.

Following several years of developing the Java Standard Edition, in late 1999 Sun Microsystems released version 1.0 of the Java 2 Enterprise Edition (J2EE) specification. From the outset, a central theme of the J2EE specification was to enable the easy development of distributed web applications. In practice, the development of J2EE applications has not been without difficulties and challenges. In many cases these have led to open source developers providing innovative solutions.

Java open source communities such as Apache Jakarta, Hibernate, Spring and JBoss together with non-Java open source communities such as Ruby on Rails and Django have played an innovative role that has contributed to ongoing discussion within the Enterprise Java community and the evolution of the Java EE specification (as it is now called) via the Java Community Process (JCP). The latest version, Java EE 5, was released in May 2006.

Example 1: Hibernate and Enterprise JavaBeans

What led to Hibernate [Hibernate]? In part, it was a J2EE project in 2001 that involved developing a web application including Enterprise JavaBeans (EJB) using eXtreme Programming practices that emphasised test driven development and continuous integration. Poor tool support for Entity EJB development and a dependency on the EJB container for running automated tests contributed to the genesis of an open source Object Relational Mapping framework that would gain significant mind share within the Java community.

Hibernate was released by Gavin King, then at Cirrus Technologies, to SourceForge in January 2002. In August 2004 the publishing of the book *Hibernate in Action* [BK04] significantly broadened the reach of Hibernate. Version 3.2 was released in October 2006.

Hibernate uses XML to configure itself as well as mappings between objects and tables (see figure 1). Via its API, Hibernate allows the programmer to concentrate on object persistence rather than coding low level JDBC (see figure 2). It has its own query language, HQL, but still allows SQL if needed. Another key benefit is the facilitation of testing outside the EJB container.

In 2006 we see the influence of Hibernate within Java EE 5 when we examine the EJB 3.0 specification and the new Java Persistence API. Hibernate founder Gavin King was a driving force behind the development of this new standards-based approach to persisting entities based on Plain Old Java Objects (POJOs).

```

<hibernate-mapping package="org.keithpitty.cricket">
  <class name="Player" table="Player" lazy="true">
    <id name="id">
      <generator class="native"/>
    </id>
    <bag name="inningses" inverse="true" cascade="save-update,lock">
      <key column="player"/>
      <one-to-many class="PlayerInnings"/>
    </bag>
  </class>
</hibernate-mapping>

```

Figure 1: Hibernate mapping XML

```

Session s = factory.openSession();
Transaction tx = s.beginTransaction();
Player player = new Player();
player.setPlayerInningses(new ArrayList());
s.save(player); // saves the Player via the Hibernate Session
// extra code here would add PlayerInningses
tx.commit();
s.close(); // releases the database connection

```

Figure 2: Example usage of Hibernate API

Example 2: Spring – A Lightweight Alternative to EJB

In 2002 Rod Johnson's book *Expert One-on-One J2EE Design and Development* [Joh02] struck a chord with developers who had experienced the painful practical limitations of developing with EJBs. Central to Rod's argument was the observation that the vast majority of J2EE applications were not distributed and hence did not warrant the complexity of EJB.

Among the motivations for the development of Spring [Spring], an application framework that utilises JavaBeans, an Inversion of Control (IoC) container and Aspect Oriented Programming (AOP) were to address areas not well served by other frameworks, allow for easy adoption, deliver ease of use, be non-invasive and make testing easier.

Spring's IoC enables the use of Dependency Injection to “wire” together JavaBeans via XML configuration files. In this way programming to interfaces is strongly emphasised and implementation classes can be easily substituted via configuration. This has particular benefits to automated testing.

```

<bean id="dataSource"
      class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <!-- properties would be defined here -->
</bean>
<bean id="sessionFactory"
      class="org.springframework.orm.hibernate3.LocalSessionFactoryBean">
  <property name="dataSource" ref="dataSource"/>
  <property name="mappingResources">
    <value>appname.hbm.xml</value>
  </property>
  <!-- more properties would be defined here -->
</bean>

```

Figure 3: Example of Spring Bean configuration

In 2006 the Spring framework continues to provide a popular “lightweight”, non-invasive alternative to J2EE with EJBs. It is commonly used in conjunction with Hibernate and various web frameworks. It has challenged Sun and the Java community to simplify the Java EE specification. EJB 3.0 has resulted but is arguably too late to attract strong mind share. Meanwhile Spring version 2.0 was released in October, 2006, as was Spring Work Flow 1.0.

Example 3: Servlets & JSPs, Struts and JSF

Struts [Struts] is “an open source framework [that] was created to make it easier for developers to build web applications based on the Java Servlet and JavaServer Pages (JSP) technologies” [Cav00]. It's creator, Craig McClanahan, donated Struts to the Apache Jakarta group of projects in 2000.

Struts attempts to put a structure in place for implementing the MVC pattern rather than leaving it to the developer to properly separate concerns into JavaBeans, JSPs and Servlets. Central to the design of Struts is the delegation from a single `ActionServlet` to one of many subclasses of the abstract `Action` class, each of which has an `execute()` method as shown in figure 4.

```
public ActionForward execute(ActionMapping mapping,
                             ActionForm form,
                             HttpServletRequest request,
                             HttpServletResponse response)
    throws Exception {
    // process input from form
    // delegate to JavaBean for business logic
    // put objects into request or session

    return mapping.findForard("success");
}
```

Figure 4: Example of Struts Action `execute()` method

Struts has become easily the most widely used web framework within the J2EE industry. Version 1.3.5 was released in September, 2006 whilst version 2, which merges the WebWork framework, is under development. Apart from WebWork, other open source web frameworks, such as Tapestry, Spring MVC and Stripes, have arisen to compete with Struts. Craig McClanahan has gone on to lead the development of the JavaServer Faces specification, version 1.2 of which is a part of the Java EE 5 specification.

Example 4: Ruby on Rails – A Reaction to J2EE Complexity?

What led to Ruby on Rails [Rails]? David Heinemeier Hansson, with a background in PHP and Java, chose Ruby as the language with which to develop a specific web application, Basecamp, in mid-2003. David came to appreciate Ruby as an “incredibly rich and expressive language” [Bla06]. From Basecamp he extracted the Ruby on Rails framework, which he released as open source in July 2004. At the time of writing Rails is in release 1.1.6.

Rails is an MVC web application framework based on Ruby. A central tenet is its “convention over configuration” philosophy. Examples of this are the standard directory structure that is generated, the naming conventions that allow implicit Active Record model to table mappings and the skeleton code for automated testing. Included in Rails is the WEBrick development server, written in Ruby, which makes it very easy to start testing quickly.

```
class Team < ActiveRecord::Base
  has_many :matches
  validates_presence_of :name
end
```

Figure 5: Example of a Rails model class

Rails has prompted similar projects such as Trails (using Java) and Grails (using Groovy). Also of note in recent times is the investment that Sun has made in JRuby. However, arguably its main contribution to the Java EE community is the debate that it has provoked. Whether it has been contrasting Active Record in Rails to Object Relational Mapping in Java,

comparing EJB 3.0 code to Active Record, the relative merits of Java EE and Rails or discussion of Ruby and scripting languages in general, Ruby on Rails has undeniably generated much controversy in the Java EE community.

Example 5: JBoss Seam and JSR 299 (Web Beans)

The Java EE 5 standard released in 2006 includes two component models: JSF 1.2 and EJB 3.0. However, they are separate models and rely on JSF backing beans to integrate JSF components and EJB components into a web application.

JBoss Seam 1.0 [Seam], released in June 2006, aims to unify the JSF 1.2 and EJB 3.0 component models. It introduces contextual components which aim to overcome session management problems so often found in J2EE stateless architectures, fix the mismatch between JSF and EJB 3.0 and enable EJB components to be bound directly to JSF views.

The following is an example of a Seam component. The highlighted annotations denote that the component is stateful, it is bound to a contextual variable called "player", it uses a persistence context associated with the `EntityManager` `em` to find and update persistent player objects and it has a beginning and end to a conversation.

```
@Stateful
@Name("player")
public PlayerBean implements Player {
    @PersistenceContext
    private EntityManager em;
    private Long id;
    public void setId(Long id) { this.id = id; }

    private Player player;
    public Player getPlayer() { return player; }

    @Begin
    public String get() {
        player = em.find(Player.class, id);
        return player == null ? "notFound" : "success";
    }

    @End
    public String save(Player player) {
        player = em.merge(player);
        return "success";
    }
}
```

Figure 6: Example of a Seam component

Not only is Seam aiming to improve upon the Java EE 5 standard, JBoss is actively participating in including its benefits in a future version of Java EE 5 via JSR 299 (Web Beans), which has been accepted. Gavin King (from the EJB 3 and Seam communities) and Kito Mann (from the JSF community) are members of the Expert Group.

Sun and Open Source Java

Sun Microsystems has slowly moved towards making Java EE and the Java Platform more open. For many years Sun has made available the source of the standard and enterprise editions of Java. However, it is only recently that Sun has released Java source under an open source licence. The source for GlassFish, Sun's Java EE 5 application server, was released in 2005 under the open Common Development and Distribution Licence (CDDL). On 13 November 2006 Sun announced [Sun06] that it was releasing Java SE, ME and EE as open source under GPLv2 [GPL] and the Java SE libraries under GPLv2 + Classpath exception [GPLCE].

Conclusion

The examples presented demonstrate how the Java EE standard has prompted innovation in the open source arena. In some cases this innovation has, in turn, led to improvements in the Java EE standard. This has provided benefits to those organisations that continue to place a high value on technologies that conform to the Java EE standard. However, the fact that the standard evolves slowly leads some developers and organisations to prefer open source solutions. My recommendation is to keep an open mind!

Resources

- [BK04] Christian Bauer and Gavin King. *Hibernate in Action*. Manning, 2004.
- [Bla06] David A. Black. *Ruby for Rails*. Manning, 2006.
- [Cav00] Chuck Cavaness. *Programming Jakarta Struts*. O'Reilly, 2000.
- [GPL] GPL v2 => <http://www.gnu.org/copyleft/gpl.html>
- [GPLCP] Classpath Exception => <http://www.gnu.org/software/classpath/licence.html>
- [Hibernate] Hibernate website => <http://www.hibernate.org>
- [JH04] Rod Johnson with Juergen Hoeller. *Expert One-on-One J2EE Development without EJB*. Wrox, 2004.
- [Joh02] Rod Johnson. *Expert One-on-One J2EE Design and Development*. Wrox, 2002.
- [Kin06] Links to audio and slides for Gavin King's presentation on Seam at TheServerSide Symposium Europe => http://www.theserverside.com/news/thread.tss?thread_id=41903
- [Rails] Ruby on Rails website => <http://www.rubyonrails.org>
- [Seam] JBoss Seam website => <http://www.jboss.com/products/seam>
- [Spring] Spring website: => <http://www.springframework.org>
- [Struts] Struts website => <http://struts.apache.org>
- [Sun06] Sun's open source announcement => <http://sun.com/opensource/java>
- [TH05] Dave Thomas and David Heinemeier Hansson. *Agile Web Development with Rails*. Pragmatic Programmers, 2005.